04-3-break-glass

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 4 deliverable: break-glass procedure and the audit trail it leaves.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`docs/03-break-glass.md`

Break-glass — production-down emergency deploy

One path. It is loud, time-boxed, and it leaves evidence the user cannot suppress.

The design principle: **people will bypass the gates during an outage whether or not I give them a way to.** If I do not build this, the bypass is someone with `AdministratorAccess` running `kubectl set image` from a laptop at 03:00, and I get no record at all. A sanctioned path that is slightly more annoying than the unsanctioned one, but visibly *supported*, is how the unsanctioned one stops being used.

When it may be used

Only for a Sev-1/Sev-2 customer-impacting production incident with a declared incident channel. Not for: a release deadline, a failing flaky gate, a waiver that lapsed on a Friday. Those are `waivers/`.

The path

`.github/workflows/break-glass-deploy.yml` (`workflow_dispatch` only) with required inputs:

input	validated how
`incident_id`	must match `INC-\d{4,}` **and** resolve to an open PagerDuty incident (API check in the first step; a fabricated id fails the job)
`justification`	free text, min 80 chars, copied verbatim into every audit record below
`image_digest`	`sha256:...` — **still by digest**. Break-glass skips the *scanners*; it does not skip signing, admission, or digest-pinning

Then:

1. `environment: break-glass`, which carries two required reviewers drawn from the on-call SRE lead group and a **0-minute wait timer** (the approval is the control, not the delay). GitHub records who approved, when, from where.
2. The blocking gates run **anyway**, with `continue-on-error: true`. They cannot stop the deploy; they *will* produce a full SARIF report attached to the run, so the post-incident review starts from data instead of memory.

3. AWS access is a **separate role**, `arn:aws:iam::123456789012:role/gha-accounts-api-breakglass`, with `MaxSessionDuration = 3600` and a trust policy whose `sub` is `repo:example-bank/accounts-api:environment:break-glass`. The normal deploy role is untouched, so a break-glass run and a normal run are distinguishable in CloudTrail by role ARN alone — not by having to correlate timestamps.
4. Session name is `breakglass-${incident_id}-${run_id}`, so every single API call made under it carries the incident id in `userIdentity.arn`.
5. Kyverno's `verify-image-signature` and `require-image-digest` policies **still apply**. There is no cluster-side bypass. An unsigned image cannot be deployed by any path, including this one. That is the line I do not move: the scanners are advisory under duress, provenance is not.

Audit trail it leaves

Six independent records, in three systems with different owners — so suppressing the evidence requires compromising all three:

record	where	who owns it	retention
Workflow run: actor, inputs, approvers, timestamps, gate SARIF	GitHub Actions	Platform	400 days (org policy)
`AssumeRoleWithWebIdentity` + every subsequent API call under `breakglass-INC-…`	CloudTrail org trail → log-archive account	Security	7 years, Object Lock
EKS control-plane audit log for the `set image`	CloudWatch → log-archive	Security	1 year
`SecurityIncidentEvent` custom EventBridge event → Security Hub finding	prod account	Security	until closed
Slack post to `#sec-incident` and `#change-control` with actor, digest, justification	Slack	Security	Slack retention
Auto-created GitHub issue `break-glass INC-xxxx` assigned to the security on-call, **72-hour SLA**	GitHub	Security	permanent

The log-archive account is a separate AWS account whose bucket has S3 Object Lock in compliance mode and an SCP denying `s3:DeleteObject` / `s3:PutBucketPolicy` to every principal including the org root. So the actor in the break-glass run — even if that actor is a compromised admin — cannot delete their own trail. That is the property that makes this evidence rather than logging.

What happens next — the part that keeps it rare

- The auto-created issue **blocks the next normal release** of `accounts-api`: `security-gates.yml` fails if an open issue labelled `break-glass` exists and is older than 72 hours. The incentive to close the loop is structural, not cultural.
- Within 72 hours, one of two PRs must merge: the actual fix (gate goes green), or a signed waiver under the normal rules (gate goes green because the finding is properly waived). "We'll get to it" is not a third option.
- Every break-glass use is a line item in the monthly risk-committee pack, with the count trend. Two in a quarter is a signal about the gates, not about the engineers — and the response is to fix the gate.

Honest limitation

Break-glass depends on the reviewers being reachable. At 03:00 on a public holiday the two-reviewer requirement is a real availability risk against a real Sev-1. The mitigation is that the reviewer group is the *on-call* SRE lead rotation, which is paged, rather than a named list of people. If the page goes unanswered for 15 minutes the honest fallback is the account's break-glass IAM user (hardware-MFA, credentials in a sealed envelope in the CISO's safe, alarm on any use) — which is outside this repo, is worse in every way, and exists because pretending it does not is how you end up with a shared admin key in a password manager instead.